

Reto 2: Sistema de Acceso con Teclado Matricial y Matriz LED 8x8

Propósito del documento:

Este documento no describe línea por línea el funcionamiento del programa, sino las estrategias y decisiones de diseño que guiaron su construcción: por qué el programa está organizado como está, qué alternativas se consideraron y qué compensaciones se aceptaron.

Filosofía general de diseño:

Todo el proyecto se apoya en tres principios:

- **Diseño cooperativo no bloqueante.** El programa avanza sobre una única base de tiempo de 1 ms y el bucle principal nunca se detiene en esperas activas. En lugar de “esperar 30 ms”, el código pregunta continuamente ¿ya pasaron 30 ms? Esto mantiene el sistema siempre atento al teclado y a la matriz.
- **Separación de responsabilidades por capas.** Cada parte del programa tiene una única función y se comunica con la siguiente a través de una interfaz estrecha. La capa de aplicación nunca toca los GPIO ni sabe de rebotes; solo reacciona a eventos limpios. La capa de presentación solo lee un buffer. Esta separación permite probar y modificar cada capa por separado.
- **Diseño dirigido por eventos y por estados.** El comportamiento se modela con dos máquinas de estado finito (una para el teclado, otra para la contraseña) que se comunican mediante eventos. Este enfoque hace explícitas todas las transiciones posibles y evita banderas dispersas y difíciles de seguir.

Estrategia de temporización: una sola base de tiempo:

Se adoptó SysTick como único reloj lógico del sistema, configurado para interrumpir cada 1 ms (SysTick_Config(16000) con reloj de 16 MHz). Cada interrupción incrementa la variable global ms.

Todos los tiempos del programa se miden con el mismo patrón, comparando contra una marca de tiempo guardada.

Ventajas de esta estrategia:

- No bloqueante: ninguna espera detiene el resto del sistema.
- Constantes centralizadas: DEBOUNCE_MS, BOX_MS y RESULT_MS reúnen todos los tiempos en un solo lugar, lo que hace el código legible y fácil de ajustar.
- Robustez: el uso de resta sin signo sobre ms, tolera correctamente el desbordamiento del contador.

Estrategia de visualización: multiplexación gestionada por interrupción:

Como no es posible controlar los 64 LEDs de forma independiente y simultánea, se emplea multiplexación por filas: en cada instante se enciende una sola fila con su patrón de columnas.

La decisión clave fue realizar la multiplexación dentro de la interrupción de SysTick y no en el bucle principal. Así el refresco ocurre a intervalos fijos e inmunes a la carga de trabajo del programa: la matriz se ve estable incluso mientras se valida la clave o se procesan pulsaciones. Con 8 filas refrescadas a 1 ms cada una, el cuadro completo se repite cada 8 ms, muy por encima del umbral de parpadeo perceptible.

Otras dos decisiones refuerzan esta capa:

- **Desacople mediante buffer:** la lógica escribe en frame[] a través de mostrar() y limpiar(), y la interrupción solo lee ese buffer. La aplicación nunca manipula directamente los GPIO de la matriz.
- **Escritura atómica con BSRR:** se usa el registro Bit Set/Reset para encender y apagar pines sin operaciones de lectura-modificación-escritura, evitando estados intermedios y parpadeos.

Estrategia de antirrebote: máquina de estados en lugar de retardo

El rebote mecánico de las teclas puede interpretarse como varias pulsaciones.

En su lugar se implementó una **máquina de estados de antirrebote no bloqueante**

(KEY_IDLE → KEY_DEBOUNCE_PRESS → KEY_PRESSED → KEY_DEBOUNCE_RELEASE) que muestrea la tecla a lo largo del tiempo usando ms. Sus ventajas:

- Depura **ambos flancos** (presión y liberación), evitando tanto pulsaciones múltiples como liberaciones falsas.
- Entrega **eventos discretos** (KEY_PRESSED_EVENT , KEY_RELEASED_EVENT) que aíslan a la lógica de la aplicación de todos los detalles de tiempo y ruido eléctrico.

- Garantiza **un solo evento por pulsación**, lo que simplifica enormemente la máquina de la contraseña.

Estrategia de lectura del teclado

El teclado se lee por barrido activo fila por fila: se pone una fila en BAJO y se leen las columnas, que tienen resistencias pull-up internas, de modo que una tecla presionada lleva su columna a BAJO.

Se aceptó de forma consciente una pequeña excepción a la regla de «no bloquear»: un breve lazo de estabilización tras cambiar de fila, necesario para que las líneas se asienten antes de leer. Se trata de unos pocos microsegundos, un costo despreciable frente a la complejidad de introducir un segundo temporizador solo para esto.

Estrategia de calibración empírica del hardware

Durante el montaje se comprobó que el orden lógico de los bits no coincide con el orden físico del cableado, tanto en las columnas de la matriz como en el teclado. La estrategia adoptada fue un mapeo dirigido por pruebas: activar un solo bit (o una sola fila/ columna) a la vez, observar qué LED o qué tecla respondía y construir a partir de ahí la tabla de correspondencia. El resultado son dos puntos que concentran toda la dependencia del cableado:

- **La función `columnas()`**, que reordena los bits antes de enviarlos a la matriz.
- **La tabla `keys[][]`**, dispuesta (transpuesta respecto a la serigrafía) para que el carácter devuelto coincida con la tecla física.

El beneficio de esta estrategia es el aislamiento: el resto del programa razona siempre en coordenadas lógicas limpias, y si en el futuro se recablea el montaje, solo hay que ajustar estos dos elementos.

Estrategia de la lógica de acceso

La lógica se modeló como una máquina de estados de cinco estados (ESPERA → INGRESANDO → VALIDANDO → ACCESO / ESTADO_ERROR) con estas decisiones de diseño:

- **Entrada de longitud fija** (4 dígitos): la validación se dispara automáticamente al contar el cuarto dígito (`n == 4`), sin necesidad de una tecla de confirmación.
- **«*» como acción universal de borrado/cancelación**: en cualquier punto del ingreso, permite al usuario recuperarse de un error sin quedar atrapado.
- **Retroalimentación visual inmediata**: el cuadro (BOX) se muestra 300 ms por cada dígito, confirmando la captura sin revelar el valor ingresado.
- **Resultado temporizado y reinicio automático**: tras 2 s mostrando la cara feliz o triste, el sistema siempre vuelve a un estado conocido (ESPERA), evitando bloqueos.
- **Comparación completa de los cuatro dígitos**: se comparan todos sin salida anticipada al primer fallo. Para este contexto no existe un requisito de resistencia a ataques de temporización, por lo que se priorizó la simplicidad y la legibilidad.

Estrategia de desarrollo incremental y verificación

El programa se construyó de abajo hacia arriba, validando cada capa antes de apoyar la siguiente sobre ella:

1. Configurar los GPIO de la matriz y encender LEDs individuales.
2. Implementar la multiplexación y verificar el refresco sin parpadeo.
3. Construir y comprobar el remapeo de columnas con patrones de prueba.
4. Definir las figuras (BOX , HAPPY , SAD).
5. Configurar el teclado y verificar `Keypad_Scan()` tecla por tecla.
6. Añadir la máquina de antirrebote y comprobar un evento por pulsación.
7. Implementar y probar la máquina de la contraseña de forma aislada.
8. Integrar todas las capas y realizar pruebas de extremo a extremo.

La razón de este orden es que los errores se localizan rápido: cuando una capa ya está validada, cualquier fallo nuevo casi siempre está en la capa que se acaba de añadir.

Estrategia de apoyo y uso de IA durante el desarrollo

La IA se utilizó como herramienta de apoyo durante el desarrollo, sin sustituir la comprobación en el hardware real. Se aplicaron estas estrategias:

- Diagnóstico por hipótesis: ante cada fallo se plantearon posibles causas y se diseñaron pruebas específicas para aislarlas.
- Desarrollo iterativo: se solicitaron ajustes pequeños y verificables, evitando modificar varias partes del programa al mismo tiempo.
- Depuración del código: se utilizó para revisar condiciones, máquinas de estados, temporización, registros GPIO y lógica de la contraseña.
- Calibración del hardware: los resultados de mediciones y pruebas con LEDs y teclado se usaron como evidencia para corregir el mapeo y el funcionamiento.
- Simplificación y documentación: se priorizaron soluciones cortas, explicables y coherentes con el código final, y la IA ayudó a organizar las decisiones en diagramas y explicaciones.